UI Toolkit Blurred Background

Table of contents

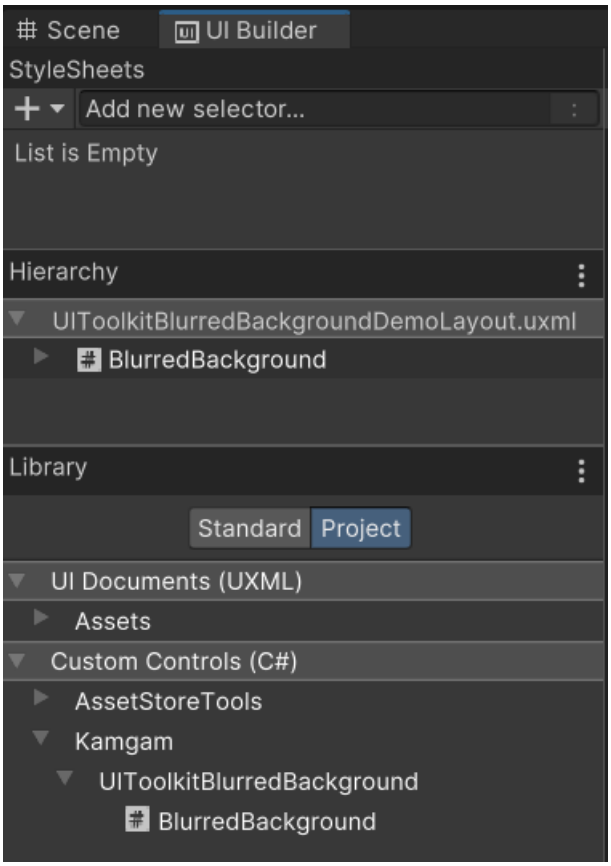
Requirements

Unity 2021.2 or higher is required since that is when Unity added the UI Toolkit Module for runtime use. If you can, please upgrade to the highest LTS version of Unity. The newer the version the less „glitches“ the UI Toolkit has.

Keep in mind, UI Toolkit as a whole is still a work in progress and not quite ready for prime time. Unity itself still recommends using UGUI instead of UI Toolkit for runtime applications (source).

Creating a blurred background

In the **UI Builder** window you can find the blurred background element under **Library > Project > Custom Controls > Kamgam > UIToolkitBurredBackground > Blurred Background**

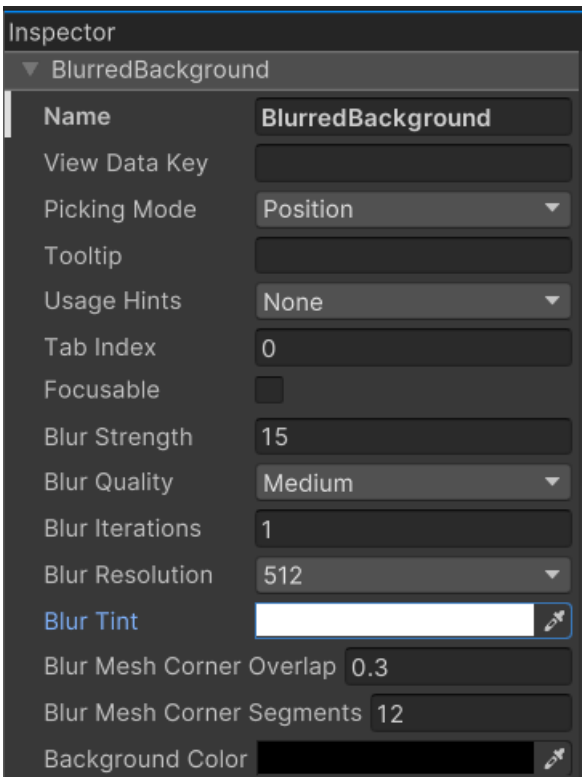


Settings

The blurred background can be controlled via the inspector in the UI Builder.

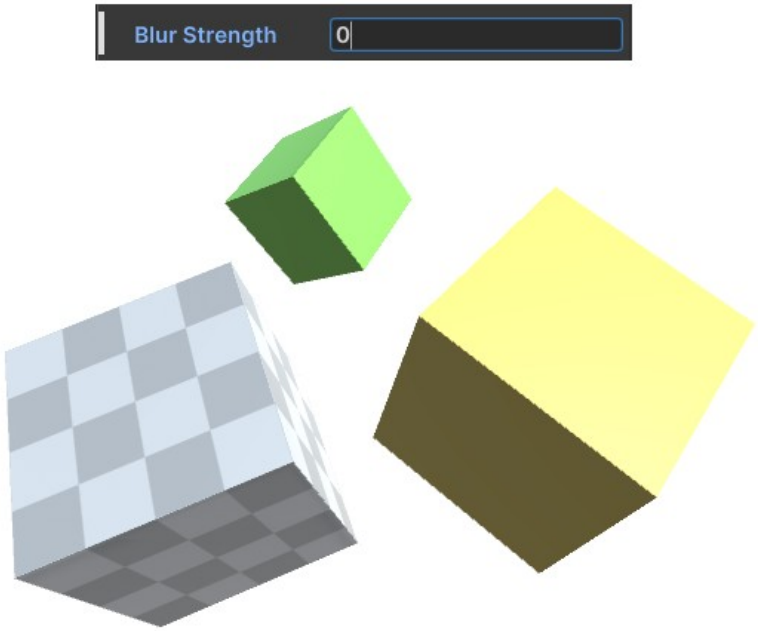
⚠ Up until v 1.3.x there only was **one global blur setting AT THE SAME TIME**. This meant that all displayed blurred backgrounds used the same blur settings.

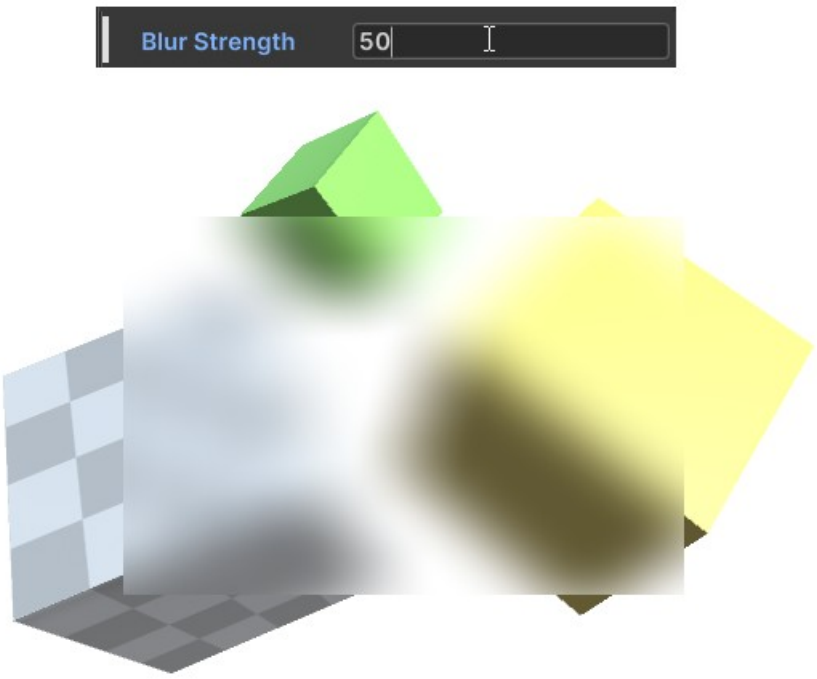
⚠ Starting with v1.4.0 each image can have individual blur strength, quality, iteration and resolution settings. I would still advise to keep the permutations on these settings low since each will spawn a new render pass (unused passes are reused though to preserve memory).


Inspector
BlurredBackground
Name
BlurredBackground
View Data Key
Picking Mode
Position
Tooltip
Usage Hints
None
Tab Index
0
Focusable
Blur Strength
15
Blur Quality
Medium
Blur Iterations
1
Blur Resolution
512
Blur Tint
Blur Mesh Corner Overlap
0.3
Blur Mesh Corner Segments
12
Background Color

Blur Strength

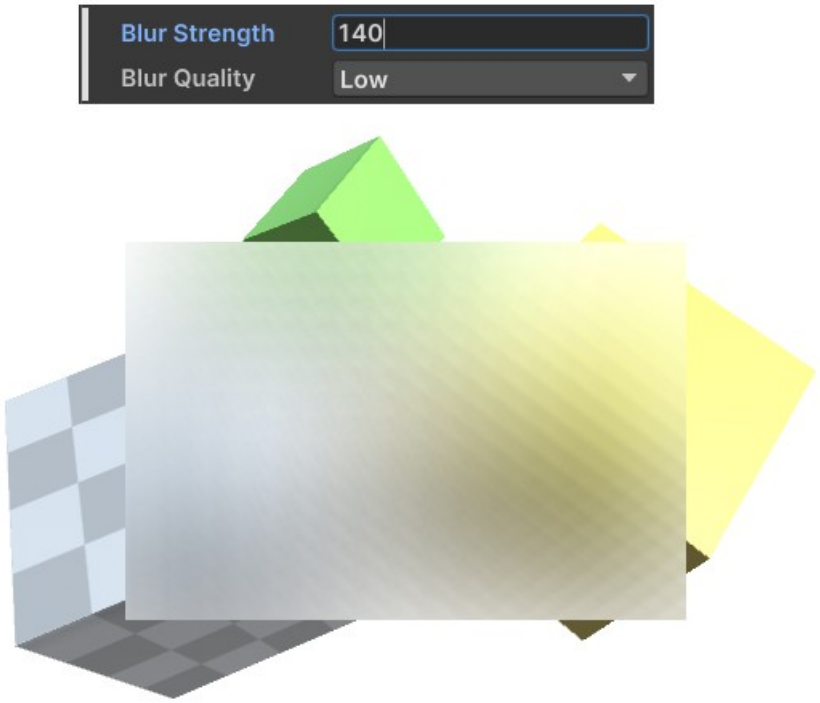
Defines how strong the blur effect will be. Notice that the quality may degrade more the higher the strength is (more on that below).

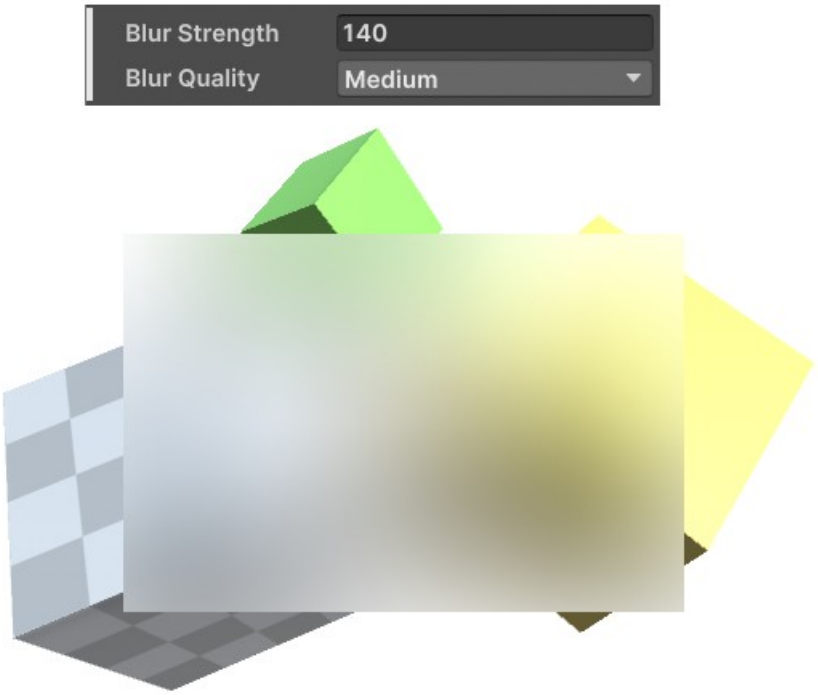



Blur Strength
50

Blur Quality

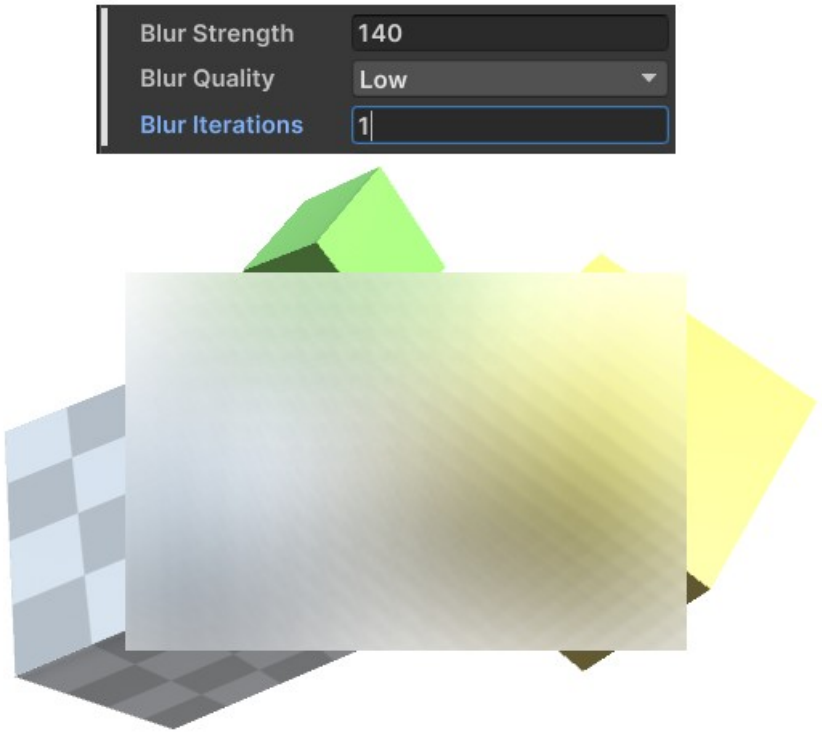
If high blur strengths are used then you may notice visible artefacts. To avoid these increase the quality. NOTICE: The higher the quality the more performance it will cost. As a rule of thumb (low = a cost of 1, medium = a cost of 3, high = a cost of 10).

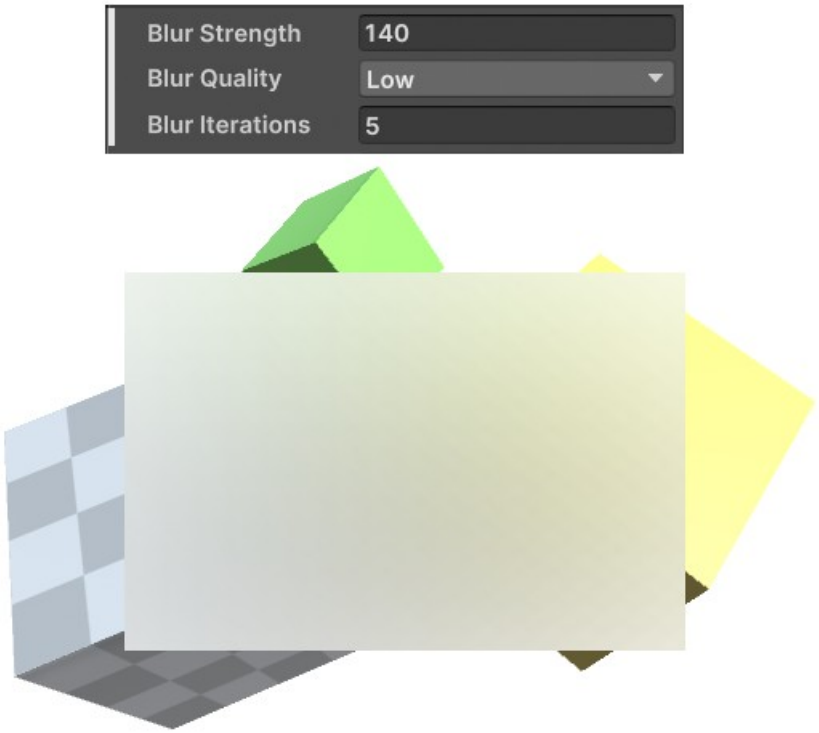




Blur Iterations

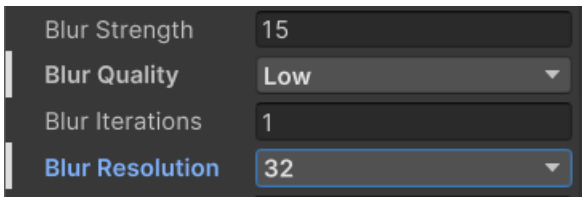
Blur iterations should be kept at 1. This defines how often the blur filter will be applied. In terms of performance this the most expensive setting you can increase. Use with care (avoid if you can).





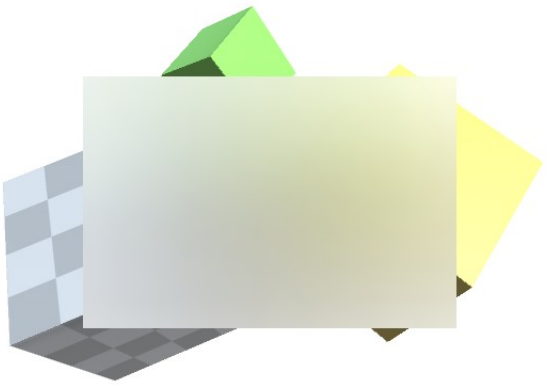
Blur Resolution

Reducing the resolution is a great way to increase the blurryness of your image while also saving a LOT of performance. Halfing the resolution usually makes the blur 4 times faster.

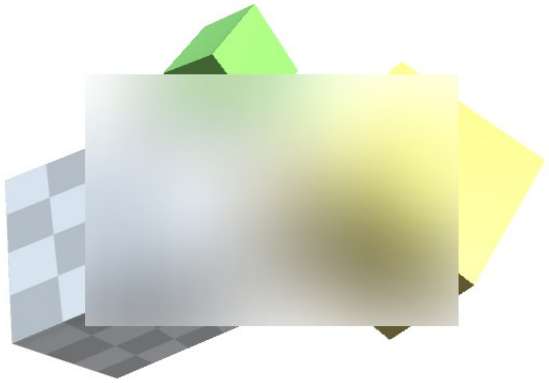


Using a low resolution can also allow you to get away with the quality set to low.

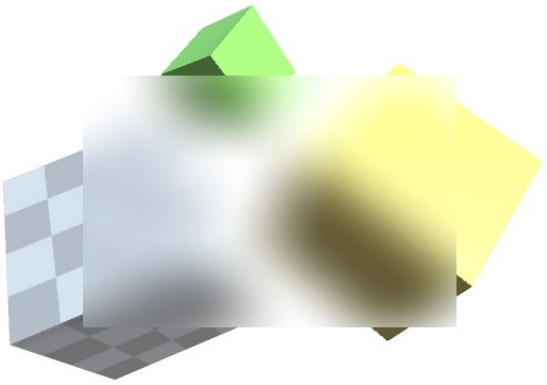
32 x 32



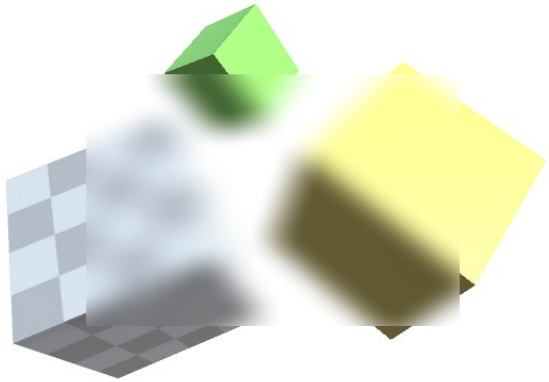
64 x 64



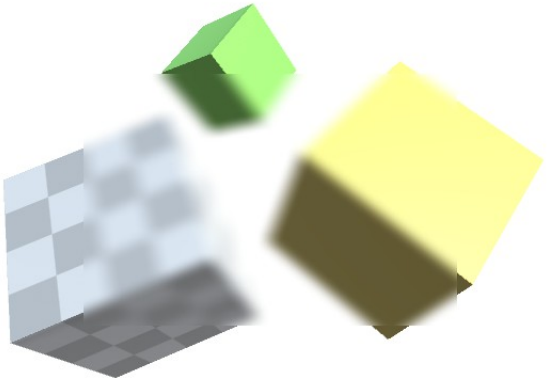
128 x 128



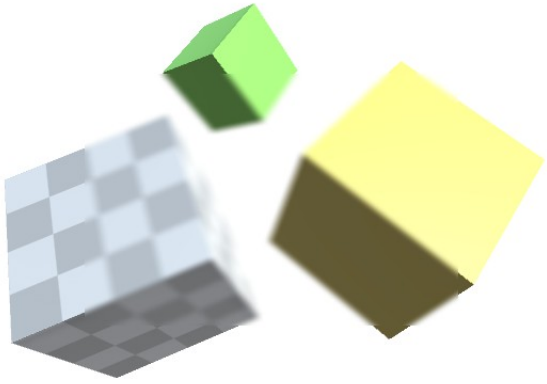
256 x 256



512 x 512

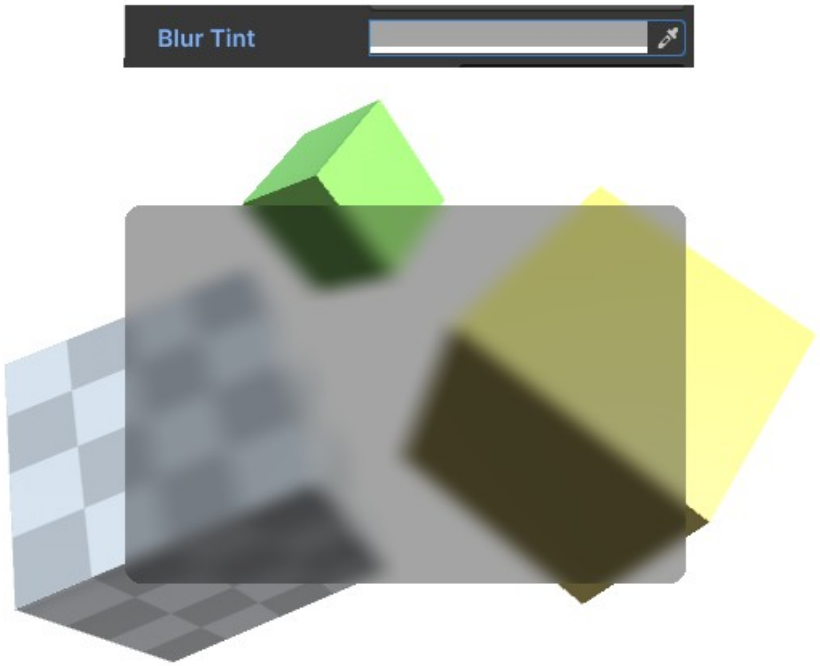


1024 x 1024

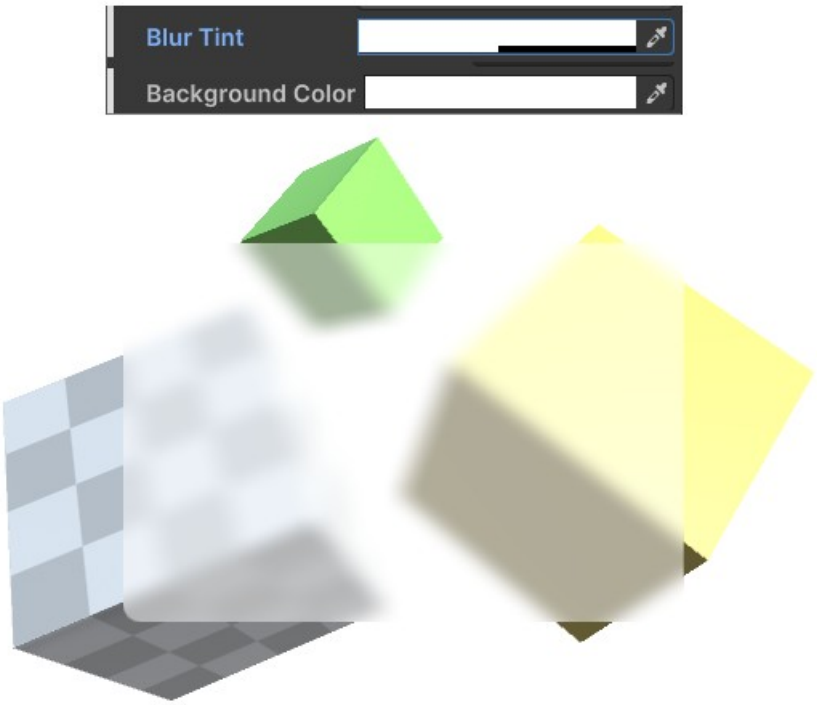


Blur Tint

By tinting the color you can make your blur look like an overlay.



You may have noticed that tinting it white will not make it more white. To achieve a milky glass like overlay you have to reduce the ALPHA of the tint and use a white, opaque color as the background. Like this:

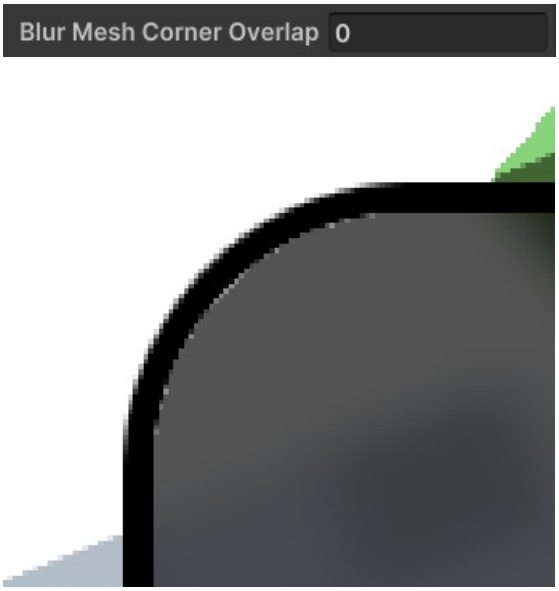


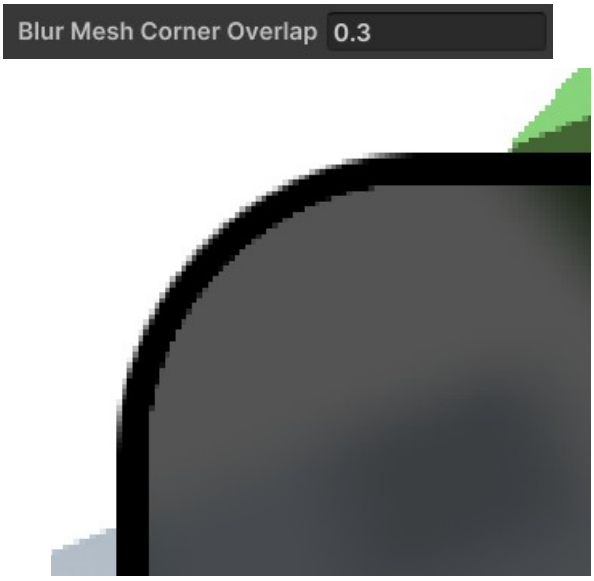
Mesh Corner Overlap

Sadly we can not (yet) modify the mesh of a visual element in UI Toolkit. That's why the blurred background has to be drawn OVER the normal mesh.

This means it has to draw a new mesh matching the borders an curves of the underlying mesh. The result of this is that sometimes there are gaps between the blurred overlay mesh and the normal border.

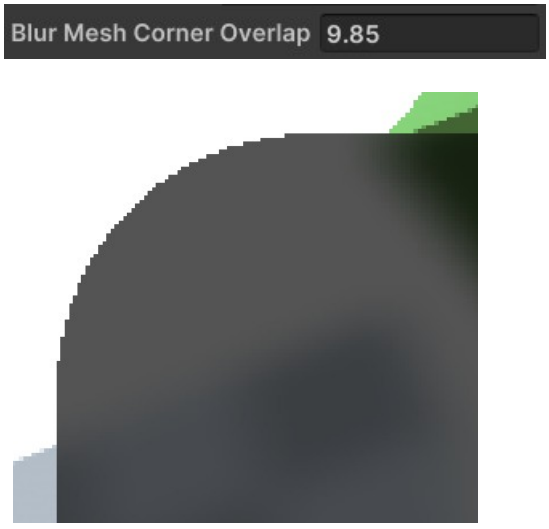
Like this (zoomed in example)





The corner OVERLAP defines how much further out the blurred mesh should be drawn in order to cover up these gaps. Usually a value of about 0.2 or 0.3 is fine.

To illustrate the process here is an image with a ridiculous overlap of about 10. Notice how all the borders are gone.



You may have also noticed that the overlay mesh edges are not anti-aliased. Sadly that's a limitation by the UI Toolkit. Hopefully they will add it in future releases.

Mesh Corner Segments

Since the blurred mesh is a new mesh on top of the default mesh it has to approximate the borders. This setting defines how detailed the rounded corners are approximated.

For illustration purposes here is a rounded corner with various segmentations:

1 2 3 4

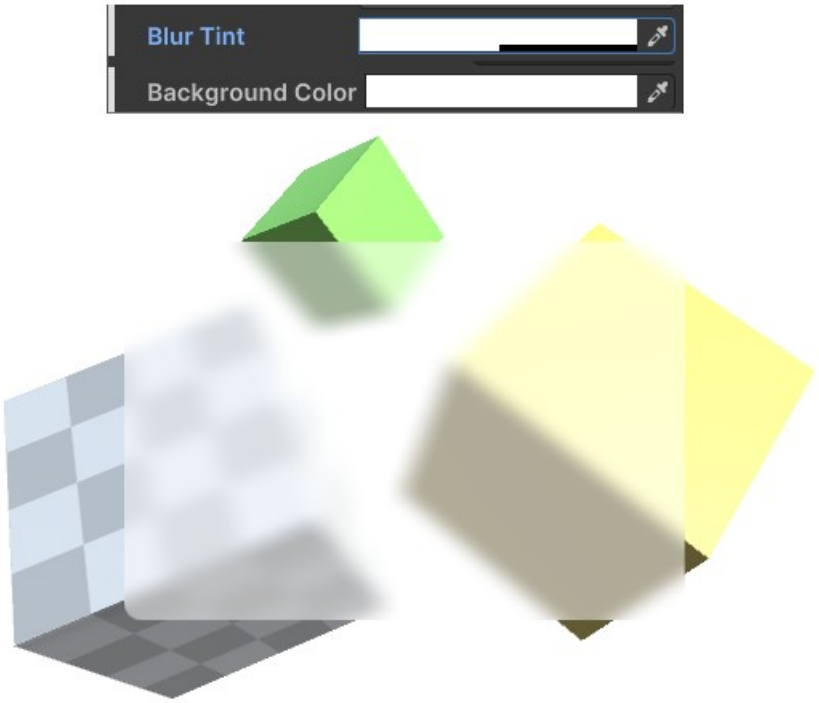
Background Color

This actually is just a wrapper for the normal background color. By default it is set to a fully transparent black color.

If you want to achieve a milky, glass like overlay you will have to reduce the alpha on the tint and use a white none transparent background color.

The blurred background image actually is an opaque image drawn on top of the normal background.

Milky glass:



There is a static "BlurredBackground.IgnoreBlurredBackgroundColor" field which you can use to disable it for ALL blurred images. If disabled you can style the background via the default background color.

USS Styles

Many properties can be controlled by USS styles too:

--blur-strength (float)

--blur-iterations (int)

--blur-quality (string, takes values „low“, „medium“ or „high“)

NOTICE: The quality values are like keywords not strings:

```
.test-styles {
    --blur-iterations: 1;
    --blur-strength: 100;
    --blur-quality: low; /* <- this works */
    --blur-quality: "high";  /* <- this does not. Please do not add any ". */
}
```

--blur-resolution (int, powers of 2 from 32 up to 2048)

--blur-tint (color)

--blur-mesh-corner-overlap (float)

--blur-mesh-corner-segments (int)

--blur-background-color (color)

UI over UI blur

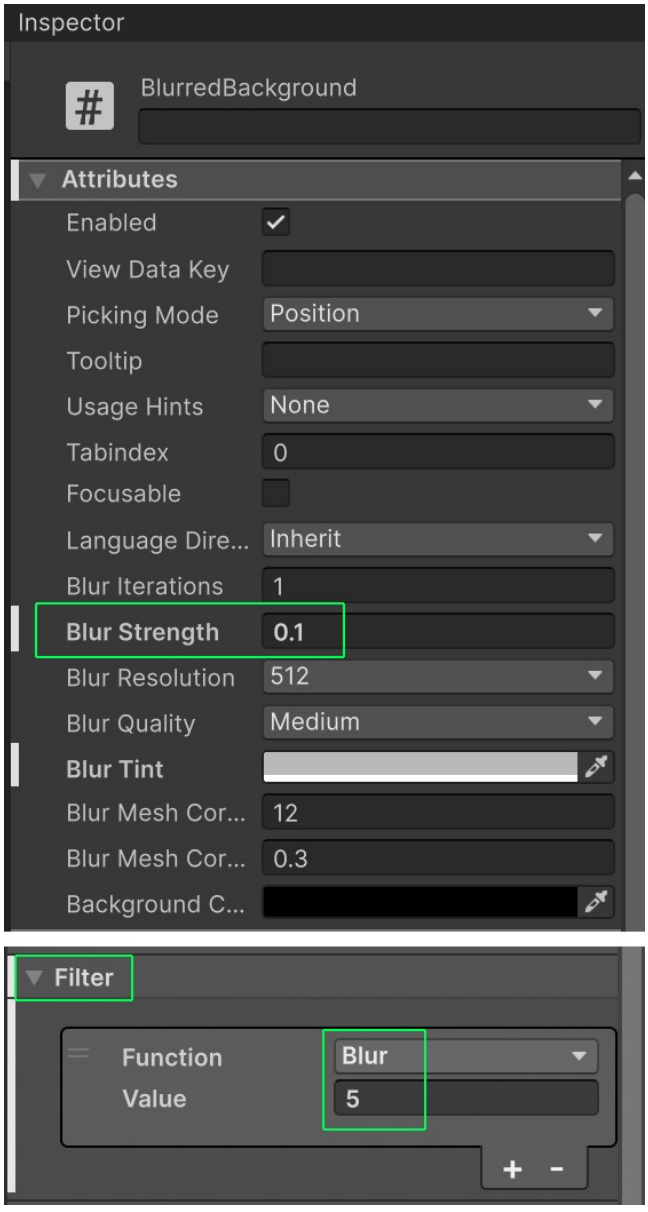
Before Unity 6.3 this simply was not possible (except in Unity 6.2+ with a mix of world and screen space UIs).

In Unity 6.3 and above Unity has finally added the ability to add filters to UI elements. However, these filters (like the UI blur) do ONLY affect the UI itself and not the scene content behind it.

This means if you want to get a fully blurred look (UI and background) you still need to use this asset. Below I will explain how to best use it in combination with filter.

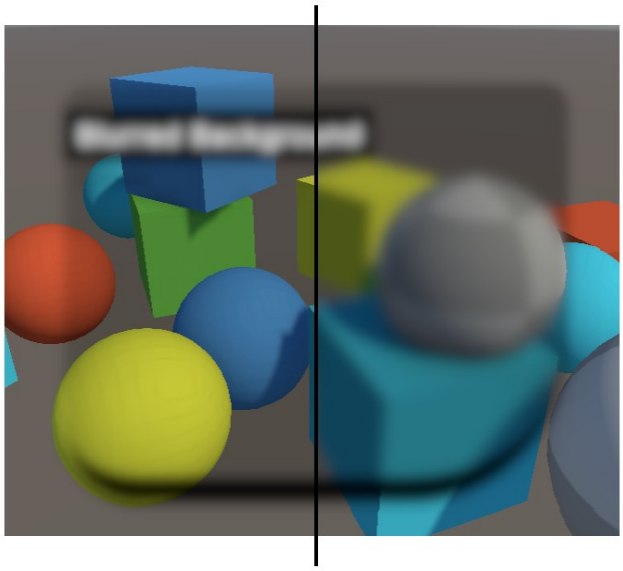
How to blur the background when using the Blur Filter in Unity6.3+

1) Use 0.1 for the blur strength on the BlurredBackground element
2) Control the blur only with the filter blur value.



Below you can see on the left an applied Blur Filter. Notice how the backgroun is not blurred.

On the right you see the result of the settings from the left. Combining a low background blur with a blur filter gives a nice blur on both.



<- HINT: You can use this low-blur trick to make any filter include the background (not just the blur) ;-)

NOTICE:

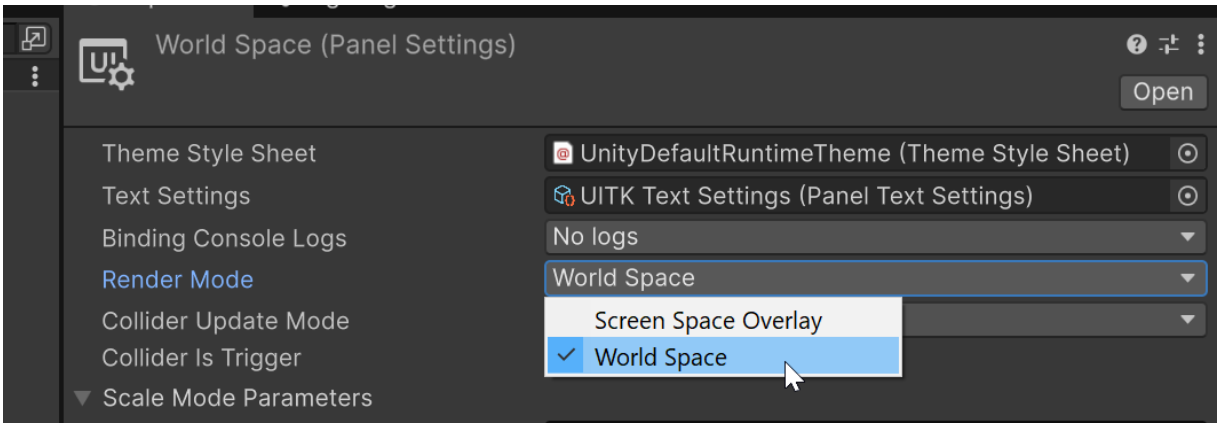
As you may notice the filters only apply to the element itself and its children, not anything that is BEHIND it. This is something I have asked Unity about (it's a limitation of the filter system) and sadly there is no ETA on when (or even if) they will change this (source). At the moment Unity does apply the filters BEFORE the scene is rendered so there is just not chance for that to be taken into account by the filters.

This means that "UI + Scene" blur is only possible for an element and it's children AND (here comes the tricky part) if that element or the children contain a BlurredBackgroundImage. Then (and only then) will the content of that background (scene) be blurred too.

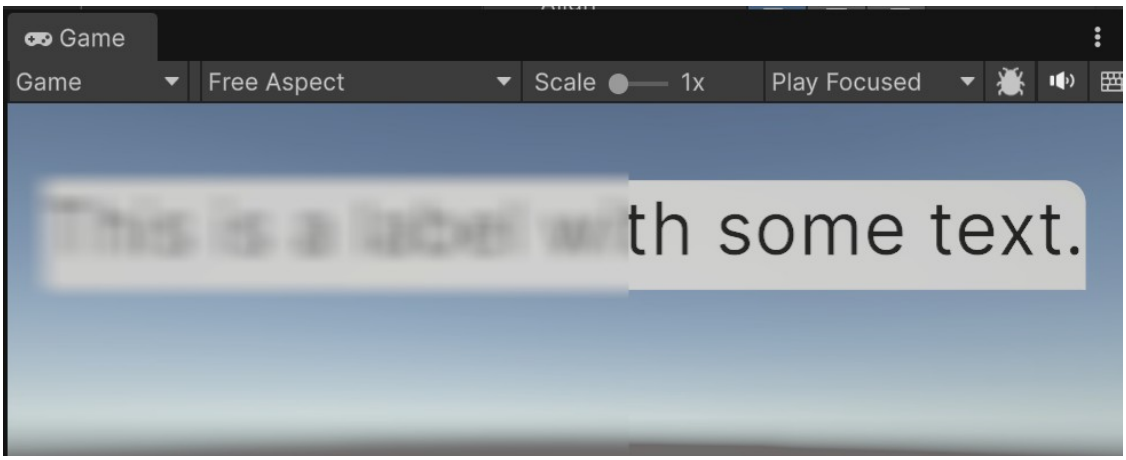
It's a side effect of the BlurredBackgroundImage containing a "blurred" copy of the scene behind it and thus (if it is a child of a filtered element) it will be taken into account by that filter. Sadly this is also the reason why any UI behind it is hidden (overwritten) by the image.

UI over UI Blur via World Space UI

Since Unity 6.2 UI Toolkit supports World Space UI. This can be used the layer the UIs with a world spave UI in the back and a screen space UI (with BlurredBackgrounds) on top.

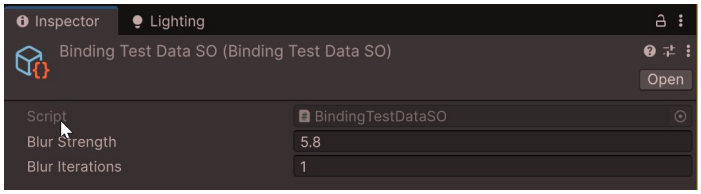


If you do this then the world space UI (text below) is blurred by the background images that are in screen space "above" it.



Data Binding

The blurred background image supports data binding just like any other visual element.



Here is a code only example:

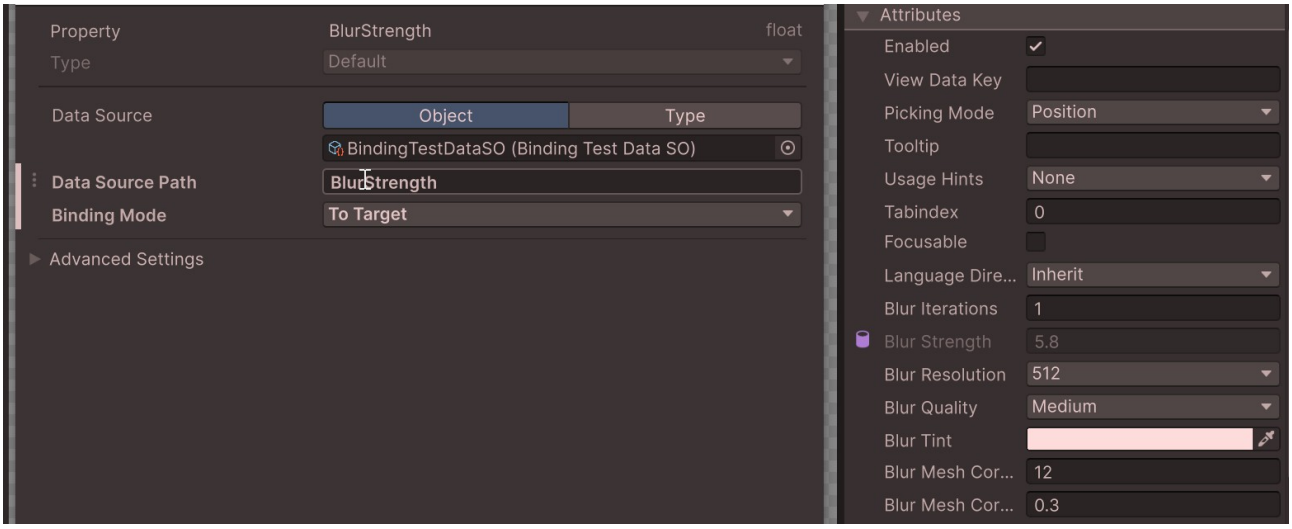
```
using Kamgam.UIToolkitBlurredBackground;
using Unity.Properties;
using UnityEngine;
using UnityEngine.UIElements;

public class BindingTest : MonoBehaviour
{
    public UIDocument Doc;
    public BindingTestDataSO Data;

    void Start()
    {
        var blurredBackground = Doc.rootVisualElement.Q<BlurredBackground>();

        blurredBackground.dataSource = Data;
        blurredBackground.SetBinding("BlurIterations", new DataBinding
        {
            dataSourcePath = new PropertyPath(nameof(Data.BlurIterations)),
            bindingMode = BindingMode.ToTarget
        });
    }
}
```

And here is how it looks like if you bind via the UI (Unity 2023.2+):



Frequently Asked Questions

Here are some common issues that have been reported.

If you can, please upgrade to the highest LTS version of Unity. The newer the version the less „glitches" the UI Toolkit has.

Keep in mind, UI Toolkit as a whole it is still a work in progress and not quite ready for prime time. Unity itself still recommends using UGUI instead of UI Toolkit for runtime applications (source).

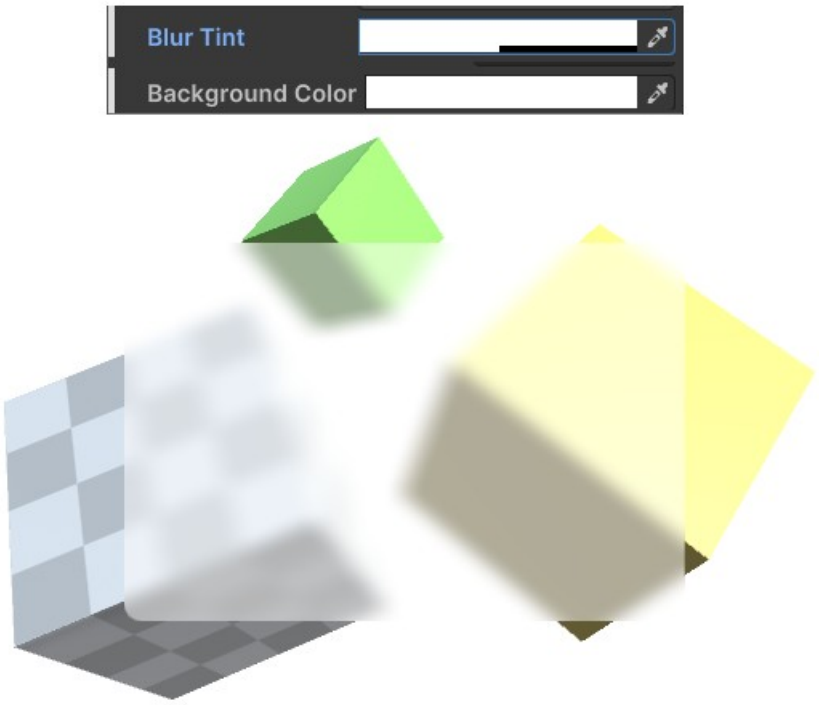
How do I make a white tint?

You will have to use the background color in combination with a tint.

Like this:

If you want to achieve a milky, glass like overlay you will have to reduce the alpha on the tint and use a white none transparent background color. The blurred background image actually is an opaque image drawn on top of the normal background.

Milky glass:



All the blurred UIs have/had the same blur settings applied.

In versions up to (including) 1.3.x that was done on purpose to save performance.

In version 1.4.0+ each image supports individual blur settings. The blur effect is generated by extracting the rendered image and blurring it. To support multiple blur factors at once it does it multiple times. The more variations of settins you have the more often it will have to add a blur pass. It is recommended to keep the variations low to save on performance. Unused variations will be repurposed to keep memory consumption in check. Ideally you still only have one combination of strength + iterations + quality + resolution active at a time (meaning all active background images use the same settings). That setup will give you the best performance (equal to pre 1.4.0 version).

The blur does not update or align properly in the game view.

It sometimes takes time for Unity to pick up on changes in the game view if not playing. However, rest assured it will work at runtime and in builds. To force an update change any of the blur settings on the element. While that's not a 100% fix it works most of the time.

The blur does not „work" in the UI Builder

That's because there is no background scene in the UI builder as reference. It will simply show the blurred background of the scene, not the background of the builder.

If „Blur Strength" or „Blur Iterations" is set to 0 then the background image vanishes.

Having either of these set to 0 would result in no blur at all and thus we do not even generate the mesh for the background override. That is done to save performance.

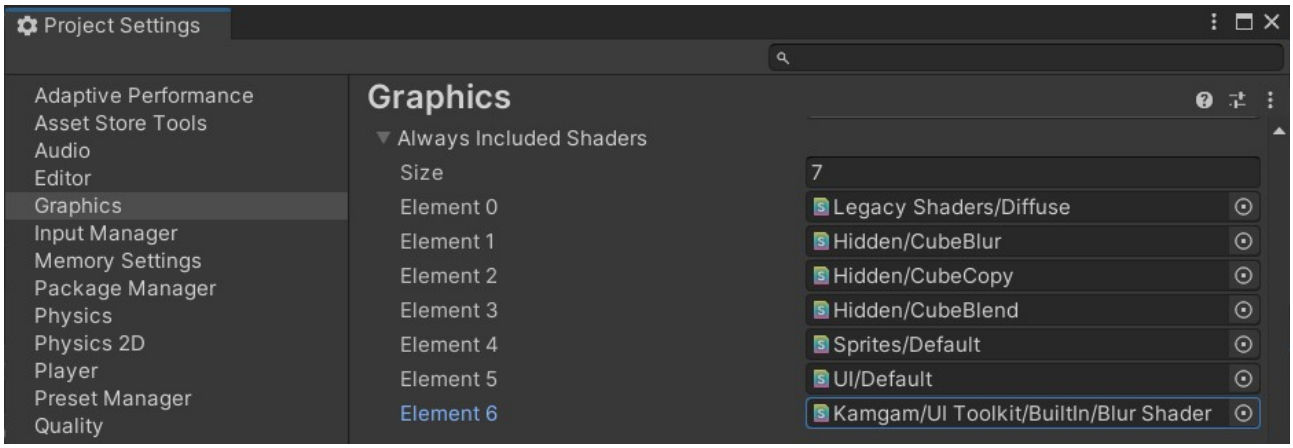
UI over another UI does not show the UI behind.

Yes, that's on purpose (it's how it works). Only scene contents are blurred and shown, sorry.

It works in the editor but not in build

Maybe you have reset the „Always included shaders". Since the blur effect is a graphics effect it is run on the graphics card and requires a shader.

Usually the shaders are added to the „Always included“ list at the start. Please check if they are added and if they are using the correct render pipeline (BuiltIn, URP or HDRP).



If I use the „transform“ to set the position then the blur does not move

Yes, sadly moving UI Toolkit elements with **transforms** is not automatically detected by UI Toolkit.

Please call .MarkDirtyRepaint() on the blurred background element after moving it with transforms. This ensures the blur position is properly updated.

HINT: Please consider moving the element with styles (left, top, margins, ..). Style changes are picked up automatically and do not require you to trigger the update manually.

I don’t like the forced square resolution of the blur. Can I change it to match my screen exactly?

v0.01 - 1.3.x:

The square resolution was done for performance reasons. I know it's not ideal for all aspect ratios (gives „artefacts“ for low blur values).

You change the resoltuion dynamically via scripts, like this:

```
using UnityEngine;

namespace Kamgam.UIToolkitBlurredBackground
{
    [ExecuteInEditMode]
    public class ChangeBlurResolution : MonoBehaviour
    {
        public bool ChangeResolution = false;
        public Vector2Int Resolution = new Vector2Int(512, 256);

        void Update()
        {
            var mgr = BlurManager.Instance;
            if (mgr != null)
                mgr.Resolution = Resolution;
        }
    }
}
```

The 512 x 256 is just some demo values. You could derive them from your current screen resolution.

v1.4.0+:

Since version 1.4.0 the resolution can (has to) be set on each image individually. While the resolution attribute only allows for square resolutions you can set the BlurResolutionSize of the image directly. NOTICE: In order for the change to stick (not being overwritten by the “blur-resolution” you have to set IgnoreSquareResolution to TRUE. This will ensure that only the BlurResolutionSize is used.

How to we handle multi camera setups?

The blur will try to find the camera to blur automatically. It first tries the main camera (the one tagged with main_camera). If that is not available (destroyed or inactive) then it tries to find the next best camera in the scene (first active camera that is not rendering in to a render texture).

Usually you should be fine if you ensure that only one camera has the main_camera tag and that this camera is active.

In HDRP if I use HDR then the blur does not work (too bright/dark)

Yes, sadly that is a problem I have not yet solved (it's complicated). As of now HDR is not supported.